

WEEK:7

1. Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

Make use of the /proc//maps file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

Code:

```
#include <stdio.h>
#include <stdlib.h>

int global_var = 10;

void code_function()
{
    printf("Address of Code:  %p\n", (void *)code_function);
}

int main()
{
    static int static_var = 20;
    int stack_var = 30;
    int *heap_var = (int *)malloc(sizeof(int));

    if (heap_var == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    *heap_var = 40;

    code_function();

    printf("Address of Global:  %p\n", (void *)&global_var);
    printf("Address of Static:  %p\n", (void *)&static_var);
    printf("Address of Heap:    %p\n", (void *)heap_var);
    printf("Address of Stack:   %p\n", (void *)&stack_var);

    free(heap_var);

    return 0;
```

```
}
```

Output:

```
adarohan18@Rohith:~$ nano MemorySegments.c
adarohan18@Rohith:~$ gcc MemorySegments.c -o MemorySegments
adarohan18@Rohith:~$ ./MemorySegments
Address of Code: 0x64158ef401c9
Address of Global: 0x64158ef43010
Address of Static: 0x64158ef43014
Address of Heap: 0x6415984e0010
Address of Stack: 0x7fffab667f6c
adarohan18@Rohith:~$ |
```

WEEK:8

Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

Code:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    int i;
```

```
    // malloc()
```

```
    int *ptr = (int *)malloc(3 * sizeof(int));
```

```
    if (ptr == NULL)
```

```
    {
```

```
        printf("malloc failed.\n");
```

```
        return 1;
```

```
    }
```

```
    printf("Memory allocated using malloc:\n");
```

```
for (i = 0; i < 3; i++)  
{  
    ptr[i] = (i + 1) * 10;  
    printf("%d ", ptr[i]);  
}
```

```
printf("\n");
```

```
// calloc()
```

```
int *cptr = (int *)calloc(3, sizeof(int));
```

```
if (cptr == NULL)  
{  
    printf("calloc failed.\n");  
    free(ptr);  
    return 1;  
}
```

```
printf("\nMemory allocated using calloc:\n");
```

```
for (i = 0; i < 3; i++)  
{  
    printf("%d ", cptr[i]);  
}
```

```
printf("\n");
```

```
// realloc()
```

```
ptr = (int *)realloc(ptr, 5 * sizeof(int));
```

```
if (ptr == NULL)
{
    printf("realloc failed.\n");
    free(cptr);
    return 1;
}

printf("\nMemory after realloc:\n");

ptr[3] = 40;
ptr[4] = 50;

for (i = 0; i < 5; i++)
{
    printf("%d ", ptr[i]);
}

printf("\n");

// free()
free(ptr);
free(cptr);

printf("\nMemory freed successfully.\n");

return 0;
}
```

Output:

```
adarohan18@Rohith:~$ nano MemoryAlloc.c
adarohan18@Rohith:~$ gcc MemoryAlloc.c -o MemoryAlloc
adarohan18@Rohith:~$ ./MemoryAlloc
Memory allocated using malloc:
10 20 30

Memory allocated using calloc:
0 0 0

Memory after realloc:
10 20 30 40 50

Memory freed successfully.
adarohan18@Rohith:~$ |
```

Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int data = 100;

    printf("Before fork:\n");
    printf("Parent PID: %d\n", getpid());
    printf("Data address: %p\n", (void *)&data);
    printf("Data value: %d\n\n", data);

    pid_t pid = fork();

    if (pid < 0)
    {
        printf("Fork failed.\n");
    }
}
```

```

        return 1;
    }

    if (pid == 0)
    {
        printf("Child Process:\n");
        printf("Child PID: %d\n", getpid());
        printf("Data address before modification: %p\n", (void *)&data);
        printf("Data value before modification: %d\n", data);

        data = 200;

        printf("\nChild after modifying data:\n");
        printf("Data address: %p\n", (void *)&data);
        printf("Data value: %d\n", data);
    }
    else
    {
        wait(NULL);

        printf("\nParent Process after child modification:\n");
        printf("Parent PID: %d\n", getpid());
        printf("Data address: %p\n", (void *)&data);
        printf("Data value: %d\n", data);
    }

    return 0;
}

```

Output:

```
adarohan18@Rohith:~$ nano forkProcMemo.c
adarohan18@Rohith:~$ gcc forkProcMemo.c -o forkProcMemo
adarohan18@Rohith:~$ ./forkProcMemo
Before fork:
Parent PID: 842
Data address: 0x7ffc31e69940
Data value: 100

Child Process:
Child PID: 843
Data address before modification: 0x7ffc31e69940
Data value before modification: 100

Child after modifying data:
Data address: 0x7ffc31e69940
Data value: 200

Parent Process after child modification:
Parent PID: 842
Data address: 0x7ffc31e69940
Data value: 100
adarohan18@Rohith:~$ |
```
